A High-Level Review

Building an Agentic AI Workflow

How to design the **agent harness** — giving an agent its context, skills, `.md` files, tools and memory — framed around Andrej Karpathy's **"LLM as operating system"** model.

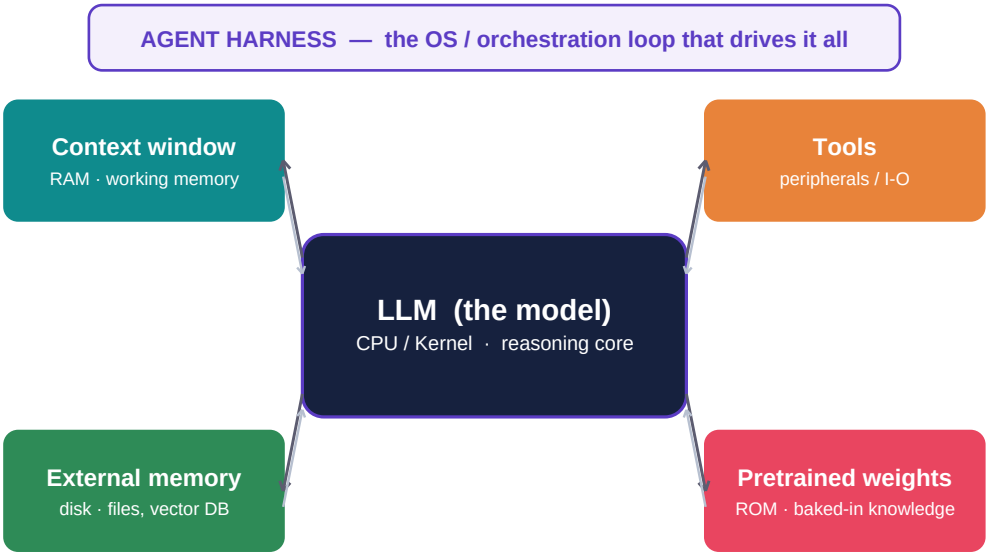

Fig. 1 — Karpathy's mental model: the LLM is the CPU, the context window is RAM, tools are peripherals, weights are ROM, and the harness is the OS.

THE ONE-SENTENCE VERSION

An **agent** is a loop where a language model is given context, decides on an action, calls a tool, observes the result, and repeats until the task is done. The **harness** is everything around the model that makes that loop reliable — and your real job as a builder is **engineering what enters the context window**.

Reference review · prepared June 2026 · sources on the final page.

1 · The mental model: the LLM is a new kind of computer

Karpathy's framing is the most useful starting point for builders. Stop thinking of the model as a chatbot and start thinking of it as the **CPU of an emerging operating system**. Everything you build around it maps cleanly onto computer architecture:

Classical computer	LLM operating system	What it means for your agent
CPU / kernel	The **model** (weights running inference)	Does the reasoning. You don't program it — you *prompt* it.
RAM	The **context window**	Finite working memory. What's in it is all the agent can ‘see’ right now.
ROM / firmware	**Pretrained weights**	Baked-in knowledge & skills. Static, always present, not directly editable.
Disk / storage	**External memory**	Files, vector DBs, history. Vast but must be *loaded* into context to matter.
Peripherals / I O	**Tools**	Code runner, browser, file system, APIs, other agents.
Operating system	The **agent harness**	The loop & plumbing that schedules turns, calls tools, manages memory.

WHY THIS MATTERS

The weights are fixed; you can't retrain them per task. Your entire lever over the system is **what you load into RAM** — the context window. That single insight is why “context engineering” is the core discipline of agent building, and why `.md` files, skills, and retrieval all exist: they are how you decide what the CPU gets to see.

2 · The agent loop — the heartbeat of every agent

Strip away the jargon and every agent is the same four-step cycle. The model never “runs” continuously; it takes one turn at a time. The harness runs the loop.

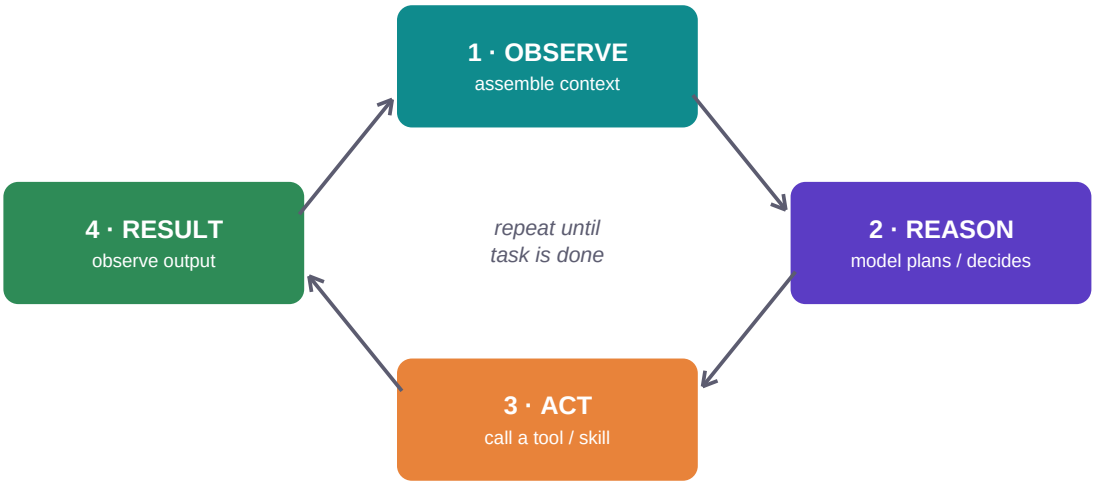

Fig. 2 — The observe → reason → act → result cycle, repeated until a stop condition is met.

- **Observe** — the harness assembles the context window: system prompt, instructions, relevant memory, the latest tool output.
- **Reason** — the model thinks and chooses the next action (often emitting a tool call).
- **Act** — the harness executes that tool/skill in the real world.
- **Result** — the output is captured and fed back in on the next turn.

DESIGN THE STOP CONDITION DELIBERATELY

A loop without a clear exit is how agents burn tokens or spiral. Common stops: task marked complete, max iterations reached, a verifier agent approves, or a human confirms. Decide this *before* you ship.

3 · The agent harness — layers around the model

The harness is everything that wraps the bare model to make it dependable. Think of it as concentric layers: closest to the core is the context the model reads; outermost is the orchestration and safety logic that governs the whole loop.

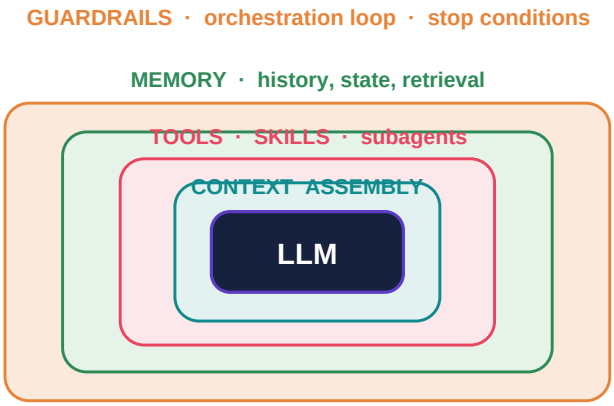

Fig. 3 — The harness as layers around the LLM core.

Each layer, briefly

- **Context assembly** — chooses what enters the window each turn (system prompt + instructions + retrieved knowledge + recent results). The highest-leverage layer.
- **Tools · skills · subagents** — the actions the agent can take and the specialized capabilities it can pull in on demand.
- **Memory** — what persists across turns and sessions: conversation history, scratchpad state, and a retrievable long-term store.
- **Guardrails & orchestration** — the loop driver, permission checks, validation of tool inputs/outputs, stop conditions, and error recovery.

4 · Context engineering — packing the agent's RAM

Because the context window is finite, building an agent is largely the craft of deciding **what to load, when, and what to leave out**. Everything competes for the same space.

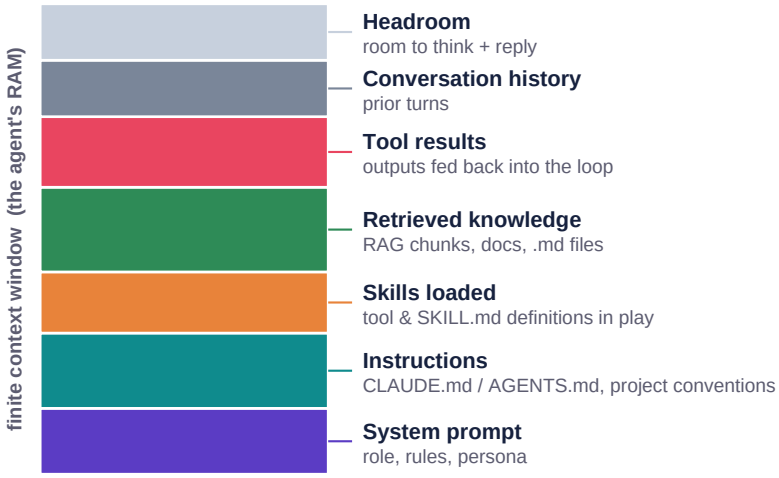

Fig. 4 — Typical composition of an agent's context window. Every segment trades off against the others and against headroom to reason.

Principles that pay off

- **Relevance over volume.** More context is not better — irrelevant tokens dilute attention and cost money. Load what this turn needs.
- **Just-in-time retrieval.** Keep big knowledge on ‘disk’ and pull only the relevant chunks into RAM when required (RAG, file reads, tool results).
- **Compaction.** As history grows, summarize older turns so the window doesn't overflow — the agent keeps the gist, not every token.
- **Stable core, dynamic edges.** Keep the system prompt and instructions fixed; vary the retrieved knowledge and tool results per turn.

5 · Skills & `.md` files — instructions the agent loads on demand

Karpathy's practical advice: **write down what the agent needs to know in plain markdown** and load it into context, rather than relying only on runtime retrieval. This is exactly what modern agent harnesses formalize as **instruction files** and **skills**.

Artifact	What it is	When it loads
`CLAUDE.md` / `AGENTS.md`	Project-level instructions: conventions, architecture, do/don't.	Every turn (small, always-on).
A **skill** (`SKILL.md`)	A packaged capability: a description + instructions + optional scripts/files.	Pulled in only when its trigger matches the task.
Reference `.md` docs	Deeper knowledge a skill or task points to.	Read on demand, as the agent needs them.
Tool / MCP defs	Schemas for the actions the agent can call.	Names always visible; full schema fetched when used.

Progressive disclosure: the key to not drowning the context

You can't load every skill and document at once — you'd blow the budget. Harnesses solve this with **progressive disclosure**: cheap metadata is always present, and the heavy content is loaded only when it becomes relevant.

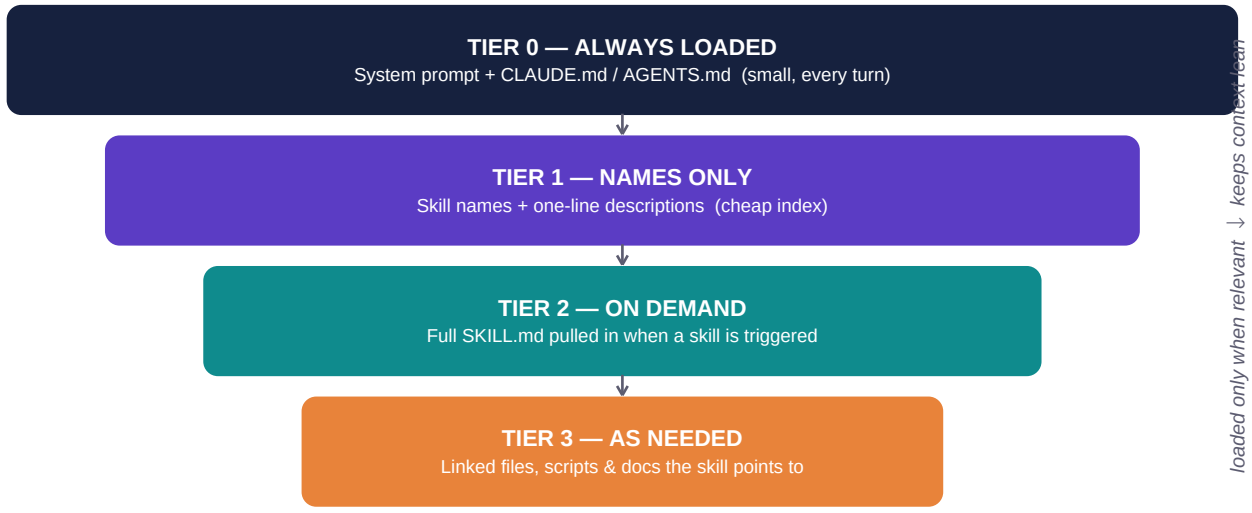

Fig. 5 — Progressive disclosure: small always-on instructions at the top, heavier capability files loaded just-in-time below.

6 · What a skill actually looks like

A skill is just a markdown file with a little front-matter. The front-matter is the cheap index the harness always sees; the body is the instruction set loaded only when triggered. Keep the description sharp — that is what decides whether the skill fires.

```
---
name: pdf-report
description: Generate a branded PDF report from a data summary.
             Use when the user asks for a PDF, export, or report.
---

# PDF Report Skill

## When to use
Trigger when the user wants a polished, shareable document.

## Steps
1. Collect the data summary and the target audience.
2. Run scripts/build_report.py with the summary as input.
3. Review the rendered output before sending.

## Files
- scripts/build_report.py   <- loaded only when this skill runs
- templates/brand.css
```

THE PATTERN, GENERALIZED

Metadata (name + description) is always cheap and always loaded → the agent knows the skill *exists*. The **body** is loaded on trigger → the agent learns *how*. Linked **scripts/files** load last → the agent acts. Same idea whether you call them skills, tools, or playbooks.

7 · Putting it together — a reference architecture

For non-trivial work, a single agent in a loop gives way to an **orchestrator** that plans and delegates to focused **subagents**, each with its own clean context, tools, and a shared memory store. This keeps any one context window small and each agent's job sharp.

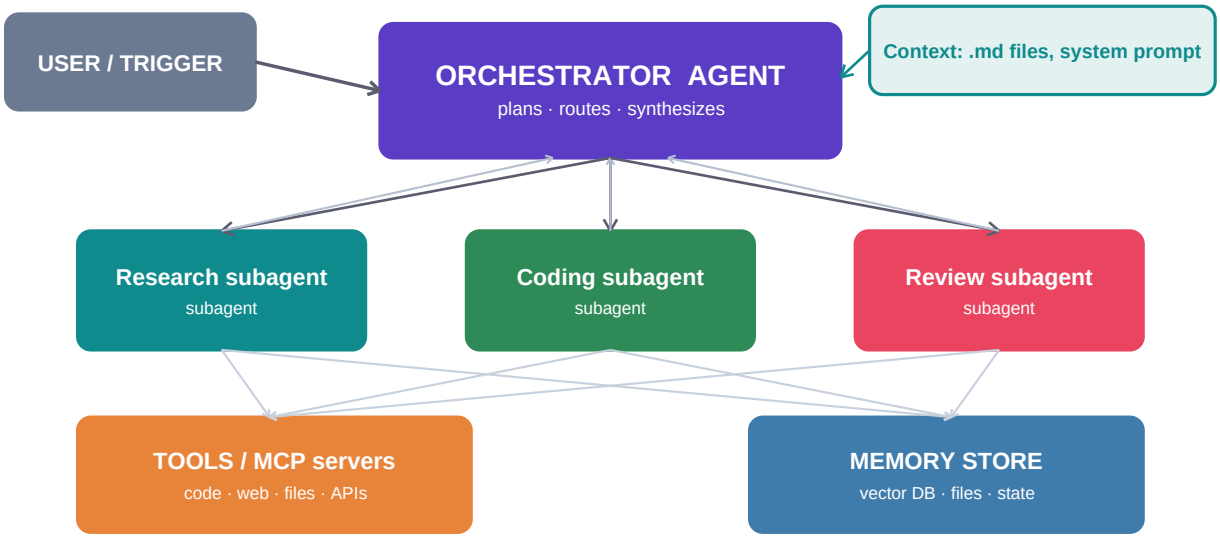

Fig. 6 — An orchestrator routes work to specialized subagents, which share tools/MCP servers and a memory store. Context (.md files, system prompt) feeds the orchestrator.

- **Orchestrator** owns the plan and the stop condition; it rarely touches raw tools itself.
- **Subagents** get a narrow brief and a fresh context window — isolation prevents one task's clutter from polluting another.
- **Shared tools / MCP** expose real-world actions uniformly to every agent.
- **Memory store** is the ‘disk’: durable state and retrievable knowledge across the run.

8 · A practical build sequence

1. **Define the job & the done-state.** One sentence on the goal; an explicit, checkable stop condition.
2. **Write the core context.** System prompt (role + rules) and a `CLAUDE.md`/`AGENTS.md` of conventions. Keep it tight.
3. **Pick the model** for the reasoning core — a current, capable model (e.g. the latest Claude) for planning; a smaller/faster one for narrow subtasks.
4. **Give it tools.** Start with the minimum set of real actions; add MCP servers for external systems.
5. **Add skills & `.md` docs** for repeatable procedures, using progressive disclosure so they load only when relevant.
6. **Wire memory.** Short-term (history + compaction) and, if needed, a long-term retrievable store.
7. **Add guardrails.** Input/output validation, permissions on risky tools, iteration caps, and a verifier where correctness matters.
8. **Split into subagents** once one context window gets crowded or jobs diverge. Introduce an orchestrator.
9. **Observe & iterate.** Log every turn; watch where context is wasted or tools misfire, and tighten.

Common pitfalls

- **Context stuffing.** Dumping everything in ‘just in case’ — it dilutes reasoning and costs tokens. Curate.
- **No stop condition.** Loops that never decide they're done.
- **Vague skill descriptions.** If the trigger text is fuzzy, the right skill never fires.
- **Tool sprawl.** Dozens of overlapping tools confuse selection — fewer, sharper tools win.
- **Treating weights as memory.** The model won't ‘remember’ your project — that's the harness's job via context and storage.

One-page recap

Model = CPU (you prompt it, not program it). **Context window** = RAM (your main lever).
Weights = ROM · **external store** = disk · **tools** = peripherals · **harness** = the OS loop.
Agent = observe → reason → act → result, until done. **Context engineering** = the core skill.
Skills & .md files load by progressive disclosure. **Scale** = orchestrator + focused subagents + shared tools/memory.

Sources

- Andrej Karpathy — “Software Is Changing (Again)” / Software 3.0, Sequoia Ascent 2026.

- Karpathy's LLM-as-OS framing: model = CPU, context window = RAM, tools = peripherals (coverage at mindstudio.ai, aibuilderclub.com, medium).
- Karpathy on writing agent knowledge as plain markdown loaded into context (LLM wiki / knowledge-base discussions).
- Practitioner write-ups on context engineering, progressive disclosure, and multi-agent orchestration.